LINEAR SEARCH IN PYTHON

Sequential Pattern Matching & Analysis

Series 08: Foundational Algorithms

Powered by: CODEHIVE EDUCATION

01. The Concept

Linear Search, also known as Sequential Search, is the most fundamental search algorithm in computer science. It is the logical starting point for understanding how a computer finds information within a collection.

The core principle is simple: the algorithm inspects every single element in a list, one by one, until it either finds the target value or reaches the very end of the collection.

When to Use It

- When your list is **unsorted** (Binary Search won't work).
- When the list is **small** and simplicity is more important than speed.
- When you are searching through data that doesn't support random access (like a basic line-by-line file stream).

Analogy: Searching for a specific book in a pile where the books are stacked randomly. You must pick up each book, check the cover, and move to the next until you find the right one.

02. Behavioral Logic

The execution of a linear search follows a strict four-step protocol:

Step-by-Step Execution:

1. **Initialization:** Start at the first element (index 0).
2. **Comparison:** Compare the current element's value with the target value.
3. **Decision:**
    - If they match, return the **current index** (Success).
    - If they don't match, move to the next index.
4. **Failure:** If you reach the end of the array without a match, return **-1** (indicates "Not Found").

This "brute force" approach guarantees finding the item if it exists, regardless of the data's order.

03. Python Implementation

Python offers two ways to perform linear searches: the built-in "high-level" approach and the custom "low-level" algorithmic approach.

The "In" Operator (Pythonic Way)

If you only need to know if an item exists, Python's native syntax is highly optimized.

```
mylist = [3, 7, 2, 9, 5, 1, 8, 4, 6]

if 4 in mylist:
    print("Found!")
```

The Custom Algorithm

To retrieve the specific position (index) of an element, you implement a loop:

```
def linearSearch(arr, targetVal):
    for i in range(len(arr)):
        if arr[i] == targetVal:
            return i
    return -1
```

04. Analysis of Time Complexity

The efficiency of Linear Search is directly proportional to the number of elements (n) in the collection.

BIG O NOTATION

O(n)

Linear Time Complexity

The Three Scenarios

- **Best Case:** The target is at the very first position.
  Time: O(1)
- **Worst Case:** The target is at the last position or not in the list at all.
  Time: O(n)
- **Average Case:** The target is somewhere in the middle.
  Time: O(n/2), which simplifies to O(n)

05. Strategic Limitations

While Linear Search is reliable, it is not "scalable." This means as your data grows, the time it takes to search grows at the same rate.

Scalability Demonstration:

Items (n)	Worst-Case Comparisons
10	10
1,000	1,000
1,000,000	1,000,000

The Conclusion: If you are working with millions of records in a sorted database, Linear Search would be painfully slow compared to Binary Search ($O(\log n)$).

06. Technical Summary

Linear Search is a fundamental building block of algorithmic thinking. It represents the baseline against which all other search algorithms are measured.

Checklist for Developers:

- Is the list unsorted? **Use Linear Search.**
- Is the dataset very small (<50 items)? **Use Linear Search.**
- Do you need the index or just a Boolean? **Choose implementation accordingly.**

CODEHIVE ACADEMY